

DOM-Graph RAG: DOM-Guided Adaptive Tiling and Budgeted Online Evidence Control for Multimodal Web-Page RAG

Alexis Firome

alexis.firome@gmail.com

Abstract. Recent visual retrieval-augmented generation (RAG) systems render documents as pixel screenshots and let a vision-language model (VLM) read tiles of the image directly, avoiding the information loss of OCR and text extraction. PixelRAG [7] popularized this idea but tiles the rendered page on a fixed pixel grid, blind to content boundaries: a table or paragraph can be sliced in half mid-tile. Separately, MAGE-RAG [9] organizes long documents into a multigranular page/element evidence graph and reads it with an online controller that activates, opens, and prunes graph nodes under a fixed action budget, but targets already-parsed long PDFs rather than rendered web pages. We present **DOM-Graph RAG**, a pipeline that replaces PixelRAG’s blind grid with tiling guided by the page’s own DOM structure: each element’s bounding box drives an adaptive tiling and patching procedure that merges undersized elements and splits oversized ones, so tile boundaries follow content boundaries by construction. The resulting tiles are organized into a MAGE-RAG-style two-level graph (page and element nodes, connected by containment and spatial reading-order edges) that we further extend across pages via hyperlink edges into a small multi-page corpus graph. We fine-tune a VLM reader contrastively on synthetically generated (tile, question) pairs with lexically-mined hard negatives, using LoRA adapters on both the vision tower and the language decoder and an InfoNCE objective. Finally, we implement an online evidence controller that evolves every node’s state through a four-state machine (INACTIVE $\rightarrow$ ACTIVE $\rightarrow$ OPENED/PRUNED) via a best-first search under a node-opening budget, so the reader receives a small, prioritized evidence subgraph instead of the whole page. We describe the full, open-sourced pipeline and its implementation choices in detail, and lay out the evaluation protocol against MAGE-RAG’s published LongDocURL and MMLongBench-Doc numbers; end-to-end benchmark results are ongoing work, reported as a work-in-progress in Section 5.

1 Introduction

Retrieval-augmented generation (RAG) over documents with rich visual layout – web pages, tables, figures, infoboxes – has long relied on text extraction: parse the HTML or run OCR, chunk the resulting text, and retrieve chunks with a dense text retriever. This pipeline discards exactly the information that layout encodes: which numbers belong to which row of a table, which caption belongs to which figure, which heading scopes which paragraph. A parallel line of work asks whether

a vision-language model (VLM) can skip text extraction altogether and read the document as an image. PixelRAG [7] shows that retrieving directly over rendered page screenshots, rather than extracted text, can match or beat text-based retrieval on long, layout-heavy documents.

PixelRAG’s tiling, however, is a fixed pixel grid applied uniformly to the rendered page, independent of what is actually on it. Independent coverage of the method notes the direct consequence: the grid “slices pages by fixed pixel height, meaning a table or paragraph can get cut in half mid-tile with no awareness of content boundaries”**[TODO: cite Venture-Beat coverage precisely, or drop this quote if a primary source is preferred]**. A retriever built on top of such tiles has to reconstruct content boundaries that the renderer already knew and threw away.

MAGE-RAG [9] addresses a related but distinct problem: given a long, already-segmented document, how should an agent decide *which* passages to actually read under a bounded reasoning budget? It builds a multigranular evidence graph over page- and element-level nodes with several explicit relations (containment, reading order, layout adjacency, section hierarchy, semantic neighbors) and an online controller (its Algorithm 1) that walks the graph, activating and pruning nodes, so that only a budgeted subset of the document is ever read by the downstream model. MAGE-RAG assumes the document is already parsed into this structure; it does not address how to obtain content-aligned tiles from a rendered page in the first place.

These two systems are complementary: PixelRAG shows that pixel tiles read by a VLM are a viable unit of retrieval for visually rich documents, but tiles them blindly; MAGE-RAG shows that a graph-structured, budget-aware controller over document elements outperforms retrieving a static top- k , but assumes the elements already exist. This paper combines them by using the one artifact PixelRAG discards and MAGE-RAG never had to construct in the first place: the DOM. When the input is a rendered web page rather than a scanned PDF, the browser’s own layout engine already knows the exact bounding box of every paragraph, heading, table, and image before a single pixel is cropped.

Contributions. We introduce **DOM-Graph RAG**, an open-source pipeline (§3) that:

1. replaces PixelRAG’s blind pixel grid with *DOM-guided adaptive tiling*: elements are cropped along their own bounding boxes, undersized elements (e.g. short list items) are merged with their vertical neighbors, over-

sized elements (e.g. a very tall table) are split into sub-tiles, heading elements are never emitted alone but always paired with the content they introduce, and elements that nest inside an already-kept element’s bounding box (e.g. an `<img>` inside a `<figure>`) are pruned before tiling rather than tiled twice (§3.1);

2. organizes these tiles into a *multigranular graph* in the style of MAGE-RAG – page and element nodes connected by containment, spatial reading-order, layout-adjacency, and section-hierarchy relations – and extends it across documents via hyperlink edges into a small corpus graph, rather than a single isolated page; a page too long to tile as one unit of work is itself *split into section-level sub-pages*, linked by a new `continues` relation and merged with the same node-namespacing machinery as the corpus graph, so that document length is handled by growing the graph rather than by excluding long documents (§3.2);
3. fine-tunes a general-purpose VLM into a page-element reader/retriever via *contrastive LoRA training* on synthetically generated (tile, question) pairs, using hard negatives mined by lexical similarity and, optionally, by a VLM judge that inspects each candidate tile directly to catch non-lexical false negatives (a paraphrase, or an answer conveyed visually rather than as text), with adapters on both the language decoder and the vision tower; the same tile set is also mirrored at reduced resolution via Lanczos downsampling, so that training can be run and compared at full and reduced resolution as a direct, controlled test of PixelRAG’s resolution/token-cost tradeoff claim (§3.3);
4. implements an *online evidence controller* that evolves each element node through an explicit four-state machine (INACTIVE, ACTIVE, OPENED, PRUNED) via a best-first search under a node-opening budget, generalizing MAGE-RAG’s Algorithm 1 to graphs built from live DOM structure rather than pre-parsed PDF layout (§3.4).

The full pipeline – rendering, tiling, graph construction (including oversized-page splitting and multi-page corpus composition), contrastive fine-tuning (including the resolution ablation), and evidence control – is implemented and exercised end-to-end on live Wikipedia pages in an accompanying notebook and a standalone batch script, with dependency-free unit tests covering the geometric, graph, mining, and image-processing logic. §5 lays out the intended evaluation against MAGE-RAG’s published LongDocURL and MMLongBench-Doc baselines; we report this as work in progress rather than fabricate numbers ahead of running it.

2 Related Work

Text-based document RAG. Standard RAG pipelines chunk extracted text (fixed-length windows, sentence or section boundaries) and retrieve chunks with a dense bi-encoder [5]. Applied to visually rich documents – PDFs

with tables and figures, web pages with infoboxes – this requires a layout-aware parser or OCR step, both of which are lossy and error-prone on complex layouts, and it discards the two-dimensional structure of the page (which text belongs to which cell, which figure a caption refers to) before retrieval ever runs.

Screenshot-based visual RAG. PixelRAG [7] sidesteps text extraction by rendering each page to a fixed-width screenshot and retrieving directly over image tiles with a VLM embedding model (Qwen3-VL-Embedding-2B in the reference implementation), following a broader trend of using VLMs as universal, modality-agnostic retrievers. Its tiling strategy is a blind, fixed-height pixel grid: simple and renderer-agnostic, but indifferent to where an element actually begins or ends on the page, so a table or paragraph can straddle a tile boundary. Our tiling procedure (§3.1) keeps PixelRAG’s core premise – a VLM reads image tiles rather than extracted text – but replaces the grid with the page’s own DOM geometry. PixelRAG separately reports image resolution as an efficiency lever, citing up to a $3\times$ token-cost reduction at lower resolutions with accuracy maintained, but specifies neither the resampling filter nor the resolutions tested in the paper or its public repository; rather than accept the claim as reported, we build a matched full-/reduced-resolution tile pair for our own dataset (§3.3) to measure the tradeoff directly under our own reader and tiling procedure.

Graph-structured evidence and online controllers for long-document QA. MAGE-RAG [9] targets long-document, multimodal QA (evaluated on LongDocURL [3] and MMLongBench-Doc [6]) by building an offline multigranular evidence graph – page and element nodes linked by containment, reading order, layout adjacency, section hierarchy, and semantic-neighbor relations – and reading it online with a budgeted controller (its Algorithm 1) that activates promising nodes, opens a bounded number of them, and prunes the rest, rather than retrieving a static top- k set once. This graph-and-controller design is the direct ancestor of our evidence controller (§3.4); the key difference is where the graph comes from: MAGE-RAG assumes an already-parsed document, whereas we construct the graph directly from the DOM of a rendered web page, which yields exact element boundaries and reading order for free but only two of MAGE-RAG’s five relation types so far (§6).

Contrastive fine-tuning and hard-negative mining. Training a bi-encoder retriever with in-batch negatives plus a handful of mined *hard* negatives – passages that are lexically or semantically close to the query but do not answer it – is standard practice in dense retrieval, e.g. DPR’s BM25-mined negatives [5]. We adopt the same idea at tile granularity, using TF-IDF cosine similarity in place of BM25 for simplicity, and generate the underlying (question, positive passage) training pairs synthetically by prompting a VLM to read each tile and produce a self-contained question, in

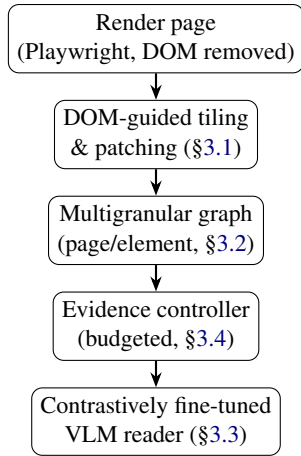


Figure 1: Pipeline overview. Rendering and tiling happen once per page; the evidence controller runs once per query, deciding which graph nodes the reader opens.

the spirit of synthetic query generation for retriever training such as InPars [1] and Promptagator [2], adapted here to image tiles rather than text passages. We fine-tune with LoRA [4] adapters applied to both the language decoder and the vision tower of the reader VLM, following the general recipe for parameter-efficient adaptation of large pretrained transformers.

Factuality benchmarks. SimpleQA [8] measures short-form factual accuracy of language models on simple, unambiguous questions; we include it in our evaluation protocol (§5) as a sanity check that evidence-controlled reading does not regress on questions that do not require multi-hop or layout reasoning.

3 Method

Figure 1 gives an overview. A page is rendered once with a headless browser; its DOM geometry drives an adaptive tiling step (§3.1); tiles are organized into a two-level graph, optionally spanning several hyperlinked pages (§3.2); a VLM reader is contrastively fine-tuned to score (question, tile) pairs (§3.3); and, at query time, an online controller walks the graph under a budget to decide which nodes the reader actually opens (§3.4).

3.1 DOM-Guided Adaptive Tiling

Rendering. We load the page in headless Chromium at a fixed viewport width (2048px) and remove interface chrome before capture – navigation bars, footers, headers, site notices, and Wikipedia-specific furniture (`.navbox`, `.mw-editsection`, ...) – via a fixed CSS selector list, then take a full-page screenshot. This mirrors PixelRAG’s rationale for stripping non-content chrome before tiling, but the removal happens in the live DOM rather than post-hoc on pixels.

Element extraction. For a configurable set of tags (headings `h1–h4`, `p`, `table`, `figure`, `img`, `ul/ol`, `blockquote`, `pre`), we read each matching element’s absolute bounding box (`getBoundingClientRect`, offset by scroll position) directly from the rendered DOM – the exact geometry the layout engine already computed, with no separate layout-analysis model needed. Hyperlinks are additionally collected from `<p>` elements only (never from tables or reference lists), each resolved to an absolute URL; these seed the cross-page edges of §3.2.

Nested-element pruning. Because several tags in that set can nest (e.g. an `<img>` inside a `<figure>`, a `<p>` inside a `<blockquote>`), a naive extraction emits overlapping bounding boxes for the same visual content – a parent element’s crop and its child’s crop covering the same pixels twice. Before tiling, we sort elements by DOM depth and drop any element whose bounding box is already fully contained (to a 1px tolerance, absorbing rendering rounding) in a shallower element already kept, so only the outermost of each nested group survives to spend a tile on that content.

Adaptive tiling and patching. Elements are processed in top-to-bottom order and turned into image crops of the full-page screenshot under three rules, so that tile boundaries follow content boundaries rather than a fixed pixel pitch:

- **Heading pairing.** A heading element is never emitted as a standalone tile – alone, it carries no answerable content. It is held pending and merged with the very next element into a single tile spanning their union bounding box (e.g. a tile tagged `h2+p` for a section header immediately followed by its first paragraph).
- **Merging (undersized elements).** Non-heading elements shorter than a minimum height (80px) are buffered and their union bounding box accumulated until the buffered span reaches that minimum, at which point the buffer is flushed as one merged tile. This avoids emitting a flood of near-empty micro-tiles for, e.g., short list items.
- **Patching (oversized elements).** An element taller than a maximum tile height (2048px, matching the render width) is sliced into $\lceil h/h_{\max} \rceil$ vertical sub-tiles of equal height, so a very tall table or figure still yields tiles a VLM can read at native resolution rather than one enormous, aliased crop.

Each tile retains a pointer back to its source DOM element(s) (tag, concatenated text preview, and any collected hyperlinks), used both for the graph’s text attributes (§3.2) and, later, as ground truth for judging whether a mined negative tile accidentally contains the answer (§3.3).

This directly targets PixelRAG’s failure mode: a table that would straddle two fixed-height grid cells is instead emitted as one tile (or patched into content-aligned sub-tiles if it is unusually tall), and a short caption is merged with its neighbor rather than left as an orphan sliver.

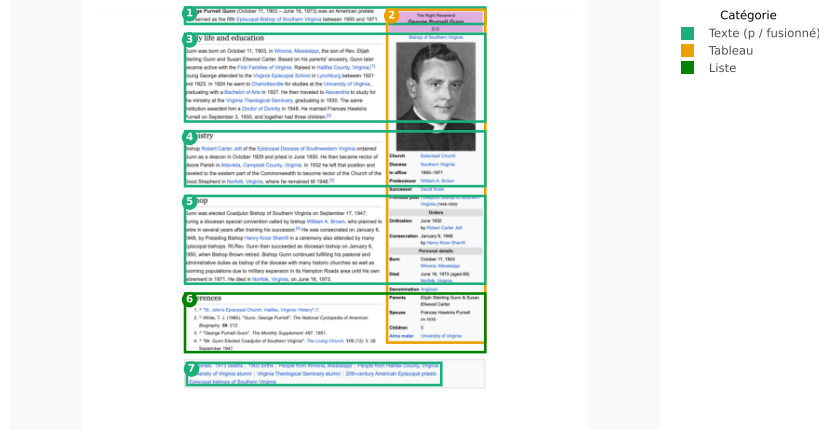


Figure 2: DOM-guided tiles (colored by content category, numbered in reading order) overlaid on the rendered page for en.wikipedia.org/wiki/George_P._Gunn. Tile boundaries follow the DOM’s own element boundaries: the infobox table (tile 2) is never split mid-row, and each section heading is paired with its first paragraph (tiles 3–6) rather than emitted alone.

3.2 Multigranular Graph Construction

Following MAGE-RAG, tiles are organized into a two-level directed graph with one `page` node and one `element` node per tile, rather than a flat list of retrieval units. We currently instantiate four of MAGE-RAG’s five relation types, plus one relation of our own for cross-page connectivity that MAGE-RAG’s single-document setting does not need:

- **contains** (`page` $\rightarrow$ `element`): every element belongs to its page.
- **reading_order** (`elementi` $\rightarrow$ `elementi+1`): a spatial reading-order heuristic, *not* raw DOM order. Elements are grouped into visual rows by vertical-overlap tolerance (20 px), each row is sorted left-to-right, and rows are concatenated top-to-bottom. This matters because DOM order and visual order diverge for CSS-positioned layouts (e.g. a sidebar infobox that appears after the lead paragraph in the DOM but is rendered beside it).
- **layout_adjacency** (`element` $\leftrightarrow$ `element`): reciprocal edges to the 2D spatial neighbors that the linear reading order flattens away – left/right between consecutive tiles of the same visual row, and above/below between a tile and its nearest horizontally-overlapping counterpart in the next row. This keeps a multi-column layout’s true neighbors (e.g. a sidebar infobox beside, not before or after, the lead paragraph) explicit in the graph rather than collapsed into a single reading-order chain.
- **section_hierarchy** (`heading` $\rightarrow$ `child`): each heading (`h1`–`h4`) points to the sub-heading or content tile immediately nested under it, respecting heading-level nesting (an `h3` attaches under the last open `h2`, not the last `h1`) to form a section tree distinct from the flat `page` $\rightarrow$ `element` `contains` relation.
- **links_to** (`element` $\rightarrow$ `external_page`): hyperlinks collected from the first $k=3$ text tiles (in reading order)

that carry any – typically the lead section – are kept as edges to placeholder external-page nodes; links beyond that budget are dropped to bound the branching factor of the corpus crawl below.

The relation still missing is *semantic-neighbor*, which requires an embedding model; see §6. Every element node is initialized with a `state` attribute set to `INACTIVE`; this is the state the online controller of §3.4 evolves at query time. It is set once, at construction time, and is otherwise inert during graph building.

Oversized-page splitting. A handful of pages are pathologically long: one Wikipedia article’s full-page screenshot exceeded 46,000 px in height and produced dozens of tiles whose sequential VLM-based QA generation (§3.3) ran for hours without completing a single example, because the whole page was one unit of work with no partial progress saved if it failed midway. Excluding such pages outright would work directly against this pipeline’s long-document motivation, so instead a page whose total element height exceeds a threshold ($10\times$ the maximum tile height of §3.1) is split into sections *before* tiling, at `h1`/`h2` boundaries – content preceding the first such heading forms a title-less preamble section – with any single section that is itself still oversized (e.g. one heading followed by an enormous table) further re-chunked by a fixed height budget as a fallback, mirroring the height-based patching already applied to individual oversized elements (§3.1), one level up. Each section is then tiled and graphed exactly as a standalone page would be, with its own `page` node and `contains`/`reading_order`/`layout_adjacency`/`section_hierarchy` subgraph, and the resulting per-section graphs are merged by node-namespacing – the same mechanism the corpus graph below uses to merge distinct pages – with consecutive sections’ `page` nodes linked by a new **continues** relation.

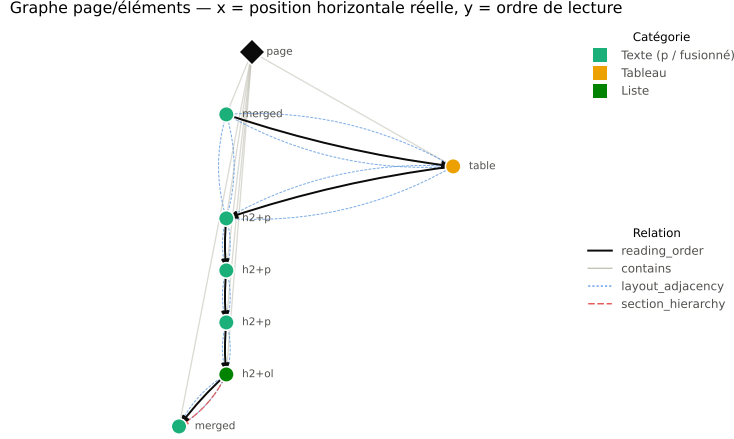


Figure 3: Page/element graph for the same page as Figure 2, laid out by real horizontal position (x) and reading-order rank (y). `reading_order` (solid black) and `contains` (light gray) are the two relations MAGE-RAG also has; `layout_adjacency` (dotted blue) and `section_hierarchy` (dashed red) are the two we add. Most headings here are paired with their content into one tile (§3.1), so `section_hierarchy` shows only where trailing non-heading content (the category footer) attaches to the last open section, rather than between the (sibling, not nested) section headings themselves.

`continues` is deliberately distinct from `links_to`: it records that two `page` nodes are sequential parts of the *same* source document, not a hyperlink to a different one, so a downstream consumer of the graph can tell "keep reading this document" apart from "jump to another document." Every page’s graph, split or not, exposes a designated *entry point* – the sole `page` node for a normal page, or the first section’s `page` node for a split one – which is what the corpus graph below rewires an incoming hyperlink onto, and is one further natural hop for a controller (§3.4) that reaches the end of one section’s evidence and still has budget left, though the controller does not yet cross `continues` edges automatically (§6).

Corpus graph. A single page’s graph can be extended into a small multi-document corpus: starting from a seed page, we crawl up to m pages reachable via its `links_to` edges (one hop, non-recursive, to keep the crawl from exploding combinatorially), build each linked page’s own graph independently – itself possibly several `page` nodes if that page had to be split, per the paragraph above – and merge them by namespacing every node with a slug derived from its page URL before composing the graphs, rewiring each seed-page placeholder external-page node onto the now-crawled page’s *entry point*. The result is a single graph spanning several pages (and, transparently, several sections of any one of them), with `links_to` edges as the only cross-document connective tissue and `continues` edges the only cross-section one; this is the substrate the evidence controller (§3.4) and, eventually, a multi-page-aware retriever would operate over.

Structured rendering. For any (sub-)graph, we serialize the `page` node and its directly contained elements to XML – one `<element id="..." tag="..."`

`state="...">text</element>` per node – as the structured multimodal input handed to the reader, in the same spirit as MAGE-RAG’s structured-rendering step. Figure 5 shows an excerpt after the controller of §3.4 has run.

3.3 Contrastive Reader Fine-Tuning

To turn a general-purpose VLM into a tile-level reader that can also score how well a tile answers a query, we fine-tune it contrastively on synthetically generated data, in three stages.

Synthetic QA generation. For every tile, we prompt the VLM with the tile image alone and ask it to produce one self-contained, factual question whose answer is visible *only* in that tile (a date, a name, a number, a caption – not a vague “what is this image about?”), or to answer `SKIP` if the tile carries no exploitable content (e.g. purely decorative). This follows the recipe of synthetic query generation for retriever training (e.g. InPars, Promptagator [1, 2]), applied here to image tiles instead of text passages. Every tile crop is persisted to disk regardless of whether it yielded a question, so that tiles which failed to produce a usable QA pair remain available as candidate hard negatives in the next stage.

Hard-negative mining. For each generated question, we rank every other tile from the *same page* by TF-IDF cosine similarity between the question and the tile’s text preview, then walk that ranking and keep the top $n=2$ that clear two false-negative filters: (a) a free lexical filter that rejects any tile whose text preview already contains the answer string verbatim (e.g. a summary tile that happens to repeat a figure from the body), and (b), optionally, a VLM-judge filter that shows the candidate tile’s image to the reader VLM itself and asks whether it can recover the same answer there – catch-

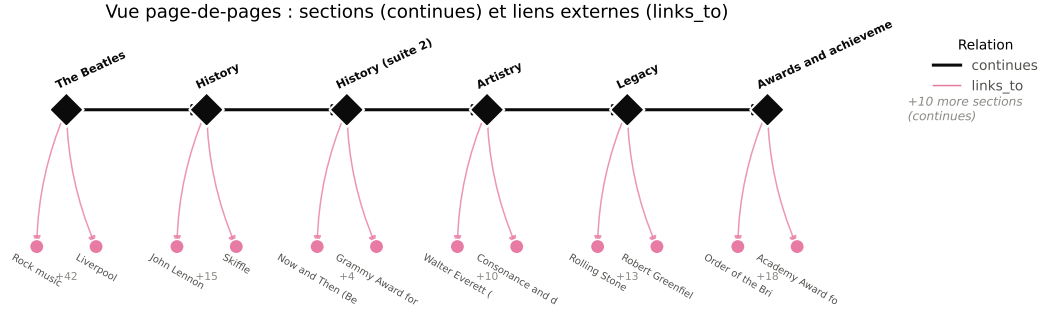


Figure 4: Page-level view of the graph for `en.wikipedia.org/wiki/The_Beatles` (280 DOM elements, split into 16 sections at h1/h2 boundaries – only the first 6 are shown for legibility). Each diamond is a page node, one per section; `continues` edges (black) chain them in document order; `links_to` edges (pink) reach external pages, collapsed here from their true element-level source (§3.2 above) to stay at page granularity. The leftmost node is the entry point.

```
<page url="..." title="...">
  <element id="tile_0000" tag="merged"
    state="inactive">...</element>
  <element id="tile_0003" tag="p"
    state="opened">Gunn was born
    on October 11, 1903...</element>
  <element id="tile_0004" tag="h2+p"
    state="inactive">...</element>
  ...
</page>
```

Figure 5: Excerpt of the XML rendering of a page graph after the evidence controller has run on the query “When and where was George Purnell Gunn born?”. Only the paragraph tile carrying the birth date/place has been opened; the rest remains inactive or pruned.

ing false negatives the lexical filter misses because the matching content is paraphrased or conveyed visually rather than as matching text (a logo, a chart’s color, a synonym). The VLM-judge filter costs one additional VLM call per surviving candidate, so it is opt-in rather than run by default. These lexically-close-but-non-answering tiles are cheaper to obtain than a learned hard-negative miner and, unlike randomly sampled negatives, are not already trivially distinguishable from the positive, which is what makes them useful contrastive signal.

Contrastive LoRA fine-tuning. Qwen2-VL is a generative, text-to-text model, not a similarity encoder. We turn it into one by prompting it with a short, fixed instruction (“summarize the above in one word”) after either the question text or a tile image, and taking the last-token hidden state of the final layer as that input’s embedding – a technique reported in recent work on repurposing generative VLMs as universal encoders[**TODO: confirm and cite the exact reference before the final version**]. We LoRA-adapt ($r=16$, $\alpha=32$, dropout 0.05) both the language decoder’s attention projections and the vision tower’s projections, rather than the decoder alone, since the task requires jointly refining what the model attends

to in the image and how it interprets the question. Given a batch of B questions, each with one positive tile and K mined hard negatives, we compute normalized question and tile embeddings and minimize a temperature-scaled ($\tau=0.05$) InfoNCE loss with cross-entropy over the $B(1+K)$ tiles in the batch – so each question’s positive competes against its own mined negatives *and*, in batch, against every other question’s positive and negatives.

Tile resolution as an efficiency lever. PixelRAG reports image resolution as a separate axis for reducing cost, citing up to a $3\times$ token-cost reduction at lower resolutions with accuracy maintained, but neither the paper nor its public repository specifies the resampling filter or the resolutions evaluated to obtain that number. Rather than take the claim at face value or guess at an undocumented implementation, we build the infrastructure to test it directly on our own tiles: every tile image can be mirrored into a resolution-reduced copy via Lanczos resampling (Pillow’s recommended high-quality downscale filter) at a configurable linear scale factor (0.5 by default, i.e. roughly $4\times$ fewer pixels), written under a separate root directory with identical filenames. Because `tiles_manifest.jsonl` and the mined contrastive examples (§3.3 above) store only tile identifiers and paths relative to that root – never pixel dimensions – the exact same synthetic questions, positive tiles, and mined hard negatives are reused unmodified for both resolutions: comparing full- and reduced-resolution training is a matter of pointing the fine-tuning run at one root directory or the other, not of re-running QA generation or mining twice. This isolates resolution as the only variable between two otherwise-identical training runs, which is what a fair token-cost/accuracy comparison against PixelRAG’s claim requires; results are reported in §5 once both runs complete.

3.4 Online Evidence Controller

Given a query, the controller of §3.2 decides which element nodes the reader actually reads, rather than handing it the

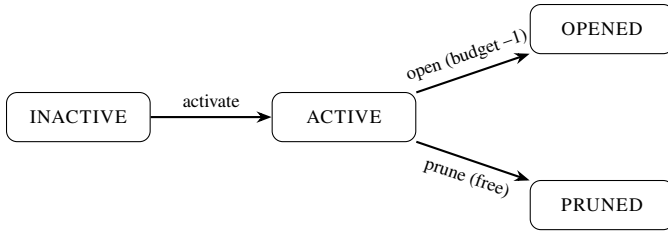


Figure 6: The controller’s state machine. Opening a node consumes one unit of the action budget and may activate unexplored `reading_order` neighbors; pruning is free and also applied to any node still ACTIVE once the budget is exhausted.

whole page (or a static top- k tile list). It evolves each element node’s state through the four-state machine of Figure 6, generalizing MAGE-RAG’s Algorithm 1 to graphs built as in §3.2.

Relevance scoring. Every element node is scored against the query by TF-IDF cosine similarity between the query and the node’s text (the same lightweight, dependency-light mechanism as hard-negative mining, §3.3, with a pure-Python token-overlap fallback if `scikit-learn` is unavailable). This is a deliberate placeholder: the natural upgrade, once the reader of §3.3 is fine-tuned, is to score nodes with its learned question–tile similarity instead of a lexical proxy (§6).

Policy. Algorithm 1 summarizes the loop. It (1) seeds the frontier by activating the k highest-scoring inactive nodes; (2) repeatedly takes the highest-scoring *active* node – pruning it for free if its score is below an opening threshold, otherwise opening it (consuming one unit of the budget B) and, only then, activating any of its still-inactive `reading_order` neighbors whose own score clears an activation threshold; and (3) once the budget is exhausted or no active node remains, prunes whatever is still ACTIVE, so that no node is left in a transient state. Because opening a node’s neighbors are only considered *after* that node has actually been opened, the frontier grows along reading order conditioned on what has already proven relevant, rather than being fixed once from a single global ranking pass – this is what distinguishes the controller from static top- k retrieval over the same relevance scores (an ablation we report in §5).

Output. The concatenation of OPENED nodes’ text, in reading order, is the final evidence passed to the downstream reader – a small, prioritized subset of the page rather than the full document, with the controller’s action log (one entry per state transition) available for inspection or visualization.

4 Implementation

The pipeline is implemented in Python and released as an open, modular package, one module per pipeline stage

Algorithm 1 Budgeted best-first evidence control

Require: graph G , query q , budget B , seed count k , thresholds $\theta_{\text{open}}, \theta_{\text{act}}$

- 1: score every element node by $\text{sim}(q, \text{text}(n))$
- 2: activate the k highest-scoring inactive nodes ▷ seed
- 3: $b \leftarrow 0$
- 4: **while** $b < B$ and some node is ACTIVE **do**
- 5: $n \leftarrow \arg \max \text{score over ACTIVE nodes}$
- 6: **if** $\text{score}(n) < \theta_{\text{open}}$ **then**
- 7: $n \leftarrow \text{PRUNED}$ ▷ free
- 8: **else**
- 9: $n \leftarrow \text{OPENED}; b \leftarrow b + 1$; record its text as evidence
- 10: **for each** inactive `reading_order` neighbor n' **of** n **do**
- 11: **if** $\text{score}(n') \geq \theta_{\text{act}}$ **then**
- 12: $n' \leftarrow \text{ACTIVE}$
- 13: **end if**
- 14: **end for**
- 15: **end if**
- 16: **end while**
- 17: prune every node still ACTIVE ▷ finalize
- 18: **return** concatenation of OPENED nodes’ text, in reading order

of §3: rendering and geometric extraction via headless Chromium (Playwright, `dom_extraction.py`); tiling, patching, and nested-element pruning over Pillow crops of a full-page screenshot (`tiling.py`); graph construction with `networkx` (`graph_builder.py`); the reader VLM served either through Ollama (qwen3-v1, the default, requiring no GPU or heavy dependencies) or through Hugging Face transformers (Qwen2-VL-7B-Instruct) when LoRA fine-tuning or direct access to hidden states is needed (`vlm_client.py`); synthetic QA generation (`qa_generation.py`); hard-negative mining with `scikit-learn`’s TF-IDF and cosine similarity, with a dependency-free token-overlap fallback if `scikit-learn` is unavailable (`hard_negative_mining.py`); and contrastive LoRA fine-tuning with `peft` and `torch`, LoRA target modules discovered by introspecting the loaded model rather than hard-coded, so the code does not silently no-op if an internal module-naming convention changes across transformers versions (`lora_finetune.py`).

Oversized-page splitting and graph composition. `sectioning.py` implements the h1/h2-boundary split and height-budget fallback of §3.2 as a pure function over already-extracted DOM elements, independent of the browser and of tiling, so it is unit-testable without Playwright. `corpus_builder.py` separates the network-bound step of a page’s pipeline (fetch + extract) from a synchronous core (`build_page_graph_from_elements`) that tiles, decides whether to split, and composes the resulting per-section graphs into one, exposing an `entry_page` attribute on every returned graph; `build_corpus_graph`

(multi-*page* composition, via `links_to`) and the splitting logic (multi-*section* composition of one page, via the new `continues` relation) share the same node-namespacing and `networkx.compose` machinery, so a linked page that turns out to be oversized is merged into the corpus graph transparently, rewired onto its entry point exactly as an ordinary page would be.

False-negative judging. The optional VLM-judge filter of §3.3 is a second pass over the candidates that already survived the lexical filter: the candidate tile’s image is shown to the reader VLM with the question and the expected answer, and a candidate is discarded if the VLM reports it can recover that answer there. It is opt-in (a `vlm_client` argument to `mine_hard_negatives`, unset by default) because it costs one VLM call per surviving candidate, on top of the one call per tile already spent on QA generation.

Resolution ablation. `image_compression.py` exposes `compress_image` (a single Lanczos resize by a linear scale factor) and `compress_images_dir`, which mirrors an entire flat tile-image directory under a new root with identical filenames; a thin CLI (`scripts/compress_dataset_images.py`) runs this over the whole dataset built so far. Because it only ever touches image bytes, and `ContrastiveTileDataset` already parameterizes its image root, switching a training run between full- and reduced-resolution tiles (§3.3) is a single `--images-root` flag to `lora_finetune.py`, with no other file to regenerate or duplicate.

Batch dataset construction. A standalone batch script (`scripts/build_dataset.py`) drives the same per-page pipeline – extraction, tiling, QA generation, and hard-negative mining – over a list of Wikipedia pages, to grow the single-page dataset used during initial development into a multi-page corpus. Page fetches retry with increasing back-off, so a transient network failure during an unattended, multi-hour run degrades to a delay rather than silently truncating the rest of the page list. The script’s unit of work is not the page but a *page unit* – one page, or one section of a page that had to be split (§3.2) – each with its own slug, generated independently, and checkpointed independently to `contrastive_examples.jsonl` as soon as it finishes; a failure partway through an oversized page therefore costs only the section in flight, not the sections already completed for that page, which matters in practice: an early version of the script lost several hours of work on a single pathologically long page (the one that motivated the splitting design of §3.2) because the whole page, not the tile, was the unit of retry.

Demonstration and tests. An accompanying Jupyter notebook runs every stage end-to-end against live Wikipedia pages, including interactive graph visualizations (state-colored, with tile images embedded in node tooltips) built

Benchmark	Metric	MAGE-RAG
LongDocURL	Accuracy	52.75
MMLongBench-Doc	Accuracy / F1	53.26 / 51.19

Table 1: Published baseline numbers from MAGE-RAG [9], the reference our evidence controller (§3.4) generalizes. DOM-Graph RAG’s results on the same benchmarks are ongoing work.

with `pyvis`. Dependency-light unit tests exercise the geometric pipeline (nested-element pruning, adaptive tiling), the graph-construction and oversized-page-splitting logic (section boundaries, the `continues` relation, entry-point resolution), hard-negative mining (both the lexical and the VLM-judge filter, the latter with a stub client), and the resolution-compression utilities, all against synthetic fixtures requiring neither network access nor a real VLM or GPU. A separate end-to-end smoke test exercises the full geometric pipeline (DOM extraction through XML export) against a bundled local HTML fixture, requiring Playwright but no external VLM or GPU either, so the core pipeline’s correctness can be checked in CI without any paid or rate-limited service.

5 Evaluation Protocol and Status

This section describes the intended evaluation and reports its current status honestly: the pipeline of §3 is implemented and exercised end-to-end (rendering through evidence control, on live pages), but we have not yet run the full benchmark suite below. We report this as work in progress rather than fill the section with numbers ahead of the actual runs.

Benchmarks. We plan to evaluate on the same long-document, multimodal QA benchmarks used by MAGE-RAG – LongDocURL [3] and MMLongBench-Doc [6] – plus SimpleQA [8] as a short-form factuality sanity check that layers of graph pruning do not regress simple, single-fact questions. Table 1 reproduces MAGE-RAG’s published numbers on the first two benchmarks as the reference to match or beat; DOM-Graph RAG’s own results on these benchmarks are pending (see status paragraph below), so no comparison row is filled in yet.

Planned ablations. Three ablations isolate each contribution of §3: (a) *tiling*: DOM-guided tiling vs. a blind fixed-height pixel grid over the same rendered pages, with the reader model held fixed, to isolate the effect of content-aligned tile boundaries; (b) *evidence control*: the budgeted controller of §3.4 vs. a static top-*k* retrieval baseline using the same relevance scores, at matched budget/*k*, to isolate the effect of best-first frontier expansion over a fixed ranking; (c) *contrastive fine-tuning*: the base VLM vs. the LoRA-fine-tuned reader of §3.3, to isolate the effect of tile-level contrastive adaptation.

Status. Phases 1–3 of §3 (rendering, tiling, graph construction and export, contrastive fine-tuning pipeline, and evidence controller) are implemented, unit- and smoke-tested, and demonstrated end-to-end on live Wikipedia pages (§4). The contrastive dataset of §3.3 was initially exercised on a single page only; we have since started scaling QA-pair generation to a small multi-page corpus with the batch script of §4 (five pages beyond the original pilot, 62 contrastive examples at the time of writing, growing as the batch continues). Scaling this generation surfaced a practical failure mode worth recording: our local reader (qwen3-v1 via Ollama) reasons at length before answering, and under Ollama’s default context window this reasoning occasionally consumed the entire budget before a final answer was emitted, silently producing an empty response that was discarded as an unusable tile rather than flagged as an error. We fixed this by widening the context window and disabling reasoning for this extraction-only step, which also matters for cost: even after the fix, QA generation on local (Apple Silicon, non-datacenter) hardware runs at roughly two minutes per tile, which is itself informative about the compute the benchmarks and ablations below will require at the scale MAGE-RAG’s published numbers were obtained on. We will complete this section, with full results tables, in a subsequent revision, most likely after moving generation and fine-tuning to rented GPU compute.

6 Limitations

Relation coverage. Our graph now instantiates four of MAGE-RAG’s five relation types (contains, reading_order, layout_adjacency, section_hierarchy), plus links_to for cross-page interconnection, which MAGE-RAG’s single-document graph does not need (§3.2). The one relation still missing is *semantic-neighbor* edges, which require an embedding model – naturally, the fine-tuned reader of §3.3 once trained at scale, rather than a separate embedder.

Arbitrary slicing of oversized elements. The patching rule of §3.1 splits an element taller than the maximum tile height into equal-height sub-tiles, with no regard for the element’s internal content (e.g. a very tall table can still be cut through the middle of a row). This is a narrower, second-order version of PixelRAG’s blind-grid failure mode that the rest of the tiling step targets: the cut boundary is only arbitrary *within* a single oversized element, rather than across the whole page, but it is not yet content-aware. Replacing fixed equal-height slicing with a dynamic, resolution-preserving patching scheme (in the spirit of Qwen2-VL’s AnyRes-style variable-resolution input) is the natural fix, avoiding both the arbitrary cut and any downscaling/distortion of the sub-tiles.

Lexical relevance scoring in the controller. The evidence controller (§3.4) scores nodes with TF-IDF cosine similarity rather than a learned embedder, so its notion of relevance is purely lexical and will miss paraphrases or cross-modal cues

that only the (image-reading) contrastively fine-tuned reader would catch. Wiring the reader’s own question–tile similarity into the scoring step is the natural next iteration, and would also let the controller reuse a single learned notion of relevance across mining (§3.3) and control (§3.4) instead of two different mechanisms.

VLM-judge false-negative filter is opt-in, not default.

As described in §3.3, hard-negative mining’s lexical false-negative filter (rejecting a candidate whose text preview contains the answer verbatim) can be supplemented with a VLM-judge filter that inspects each candidate’s image directly, catching non-lexical false negatives (a paraphrase, or an answer conveyed visually rather than as text). Because it costs one additional VLM call per surviving candidate, it is not run by default, and the contrastive dataset built so far (§5) does not yet use it – so it remains exposed to exactly the class of false negative it was built to catch until a run is repeated with it enabled.

Single-page scope for mining and (mostly) for control.

Hard-negative mining operates within one page at a time, since tile identifiers are unique only at page scope; mining negatives *across* the pages of a corpus graph (§3.2) is a natural extension that would require reusing the same slug-prefixing scheme used to merge per-page graphs. The evidence controller itself already operates over whatever graph it is given, including a multi-page corpus graph, but we have not yet evaluated it at that scale, nor extended its neighbor-propagation step to cross the links_to relation into newly-activated pages.

Shallow, non-recursive crawl. The corpus graph follows hyperlinks one hop deep from a seed page, bounded to a handful of linked pages, to avoid the combinatorial blow-up of a recursive web crawl; this is sufficient to exercise cross-page graph structure but is far short of the scale a genuinely multi-document retrieval setting would require.

7 Conclusion

We described DOM-Graph RAG, a pipeline that grounds two recent ideas in visual and graph-structured RAG – PixelRAG’s screenshot tiles and MAGE-RAG’s budgeted graph controller – in the one source of structure a rendered web page already provides for free: its DOM. DOM-guided adaptive tiling replaces a blind pixel grid with content-aligned tiles; a multigranular page/element graph, extendable across hyperlinked pages, replaces a flat tile list; a contrastively fine-tuned VLM reader replaces a generic off-the-shelf one; and a four-state, budget-bounded evidence controller replaces retrieving either the whole page or a static top- k . Every stage is implemented, tested, and demonstrated end-to-end on live pages. The immediate next step is Phase 4: running the LongDocURL, MMLongBench-Doc, and SimpleQA evaluation and the three ablations laid out in §5, and, in parallel,

closing the gaps noted in §6 – most importantly, replacing the controller’s lexical relevance scoring with the fine-tuned reader’s own learned similarity once it is trained at scale.

References

- [1] Luiz Bonifacio, Hugo Abonizio, Marzieh Fadaee, and Rodrigo Nogueira. Inpars: Data augmentation for information retrieval using large language models, 2022.
- [2] Zhuyun Dai, Vincent Y. Zhao, Ji Ma, Yi Luan, Jianmo Ni, Jing Lu, Anton Bakalov, Kelvin Guu, Keith B. Hall, and Ming-Wei Chang. Promptagator: Few-shot dense retrieval from 8 examples, 2022.
- [3] Chao Deng et al. Longdocurl: a comprehensive multi-modal long document benchmark integrating understanding, reasoning, and locating, 2024. Author list and venue to verify precisely before final submission.
- [4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- [5] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wentaoh Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [6] Yubo Ma, Yuhang Zang, et al. Mmlongbench-doc: Benchmarking long-context document understanding with visualizations, 2024. Author list to verify precisely before final submission.
- [7] Yichuan Wang et al. Pixelrag: Web screenshots beat text for retrieval-augmented generation, 2026. UC Berkeley / BAIR / Berkeley NLP. Code: github.com/StarTrail-org/PixelRAG. Author list as listed on arXiv; verify full author list before final submission.
- [8] Jason Wei et al. Measuring short-form factuality in large language models, 2024. OpenAI. Author list to verify precisely before final submission.
- [9] Yilong Zuo et al. Multigranular adaptive graph evidence for agentic multimodal rag in long-document qa, 2026. Code: github.com/laonuo2004/MAGE-RAG. Author list as listed on arXiv; verify full author list before final submission.